

AI Security Audit Prompts

Essential prompts for securing AI-generated web applications

When using AI tools (like Bolt, Lovable, or v0) to build applications, they often prioritize functionality over security. Use these specialized, high-impact prompts to lock down security, protect user data, harden business logic, and prepare your app for production.

1. The Database Security & Privacy Lockdown

AI app builders often set database access rules to "public" by default so things work instantly during testing. This prompt seals those leaks.

"Act as a Lead Database Security Architect. Audit my entire database schema and connection setup. Specifically check for:

1. Missing or overly permissive Row Level Security (RLS) policies. Verify that User A can never read, update, or delete User B's data.
2. Database credentials or connection strings exposed to the frontend.
3. Missing data sanitization against SQL injection or unexpected data types.

Provide a list of vulnerabilities found, then refactor the schema definitions and API calls to enforce strict data isolation."

2. Authentication & Session Security Audit

User login bugs are common in AI-generated code, leading to hijacked sessions or account takeovers.

"Act as an Identity and Access Management (IAM) Specialist. Audit the authentication flow, session management, and routing logic of this app. Check for:

1. Unprotected frontend routes or components that render sensitive data before verifying auth state.
2. Weak JWT token storage (e.g., storing raw auth tokens in plain localStorage instead of HTTP-only secure cookies).
3. Flaws in password reset, token expiration, or logout invalidation.

Highlight any flaws and update the auth context/middleware to enforce secure session practices."

3. Payment & Billing Logic Protection (Stripe / Webhooks)

If you charge users money, AI code often misses server-side verification, allowing tech-savvy users to bypass paywalls or alter prices.

"Act as a Fintech Security Engineer. Audit all payment handling and Stripe/webhook integrations in this code. Specifically verify:

1. Prices and subscription tiers are calculated strictly on the backend/server side, never trusted from frontend inputs.
2. Webhook endpoints validate raw request signatures to prevent forged payment notifications.
3. Idempotency handling (preventing double-charging if a payment request is submitted twice).

Identify any logic bypasses and patch the code to secure the checkout and webhook pipelines."

4. Input Validation & Injection Hardening

Gives your app defense against malicious inputs, XSS (Cross-Site Scripting), and malicious file uploads.

"Act as an Application Security Penetration Tester. Review all forms, search bars, file uploaders, and API request handlers in this app. Specifically inspect for:

1. Cross-Site Scripting (XSS) risks where user-submitted text or HTML is rendered un-sanitized.
2. Missing server-side schema validation (e.g., using Zod or Yup) on incoming API payloads.
3. Unrestricted file upload vulnerabilities (missing file-type, extension, and size checks).

Refactor the input handlers to include strict sanitization and validation schemas."

5. Rate Limiting & Abuse Prevention

Prevents bots from spamming your APIs, draining your AI tokens, or causing massive server bills.

"Act as a Cloud Infrastructure & Security Engineer. Audit this application for API abuse, denial-of-service (DoS) risks, and resource exhaustion. Check if:

1. Public or expensive API routes (especially AI model prompts or email triggers) lack rate limiting.
2. Unauthenticated users can trigger backend functions repeatedly.
3. Batch endpoints lack pagination or item limits.

Show me where abuse is possible and write middleware/rate-limiting logic to cap requests per IP or authenticated user."

How to Use These Prompts Effectively

- **Run them one at a time:** Don't throw all five prompts at the AI at once. Apply them sequentially as your app nears completion.
- **Ask for explanations first:** Add this to the end of any prompt: *"Before giving me the updated code, give me a 3-bullet-point summary of the exact security bugs you found and fixed."*
- **Re-test the app immediately:** Security fixes can sometimes break features that relied on loose permissions. Always click through your user flows right after applying a security patch.

